Reverse Engineering Challenges

Chal1

פרמטרים:

אחרי נסיון הרצה הבנתי כי צריך 5 פרמטרים.

```
user36@netanya-students:~$ ./chal1
please try again and make sure to give the correct amount of arguments („˘ᴗ˘„)
user36@netanya-students:~$ ./chal1 1
please try again and make sure to give the correct amount of arguments („˘ᴗ˘„)
user36@netanya-students:~$ ./chal1 1 2
please try again and make sure to give the correct amount of arguments („˘ᴗ˘„)
user36@netanya-students:~$ ./chal1 1 2 3
please try again and make sure to give the correct amount of arguments („˘ᴗ˘„)
user36@netanya-students:~$ ./chal1 1 2 3 4
please try again and make sure to give the correct amount of arguments („˘ᴗ˘„)
user36@netanya-students:~$ ./chal1 1 2 3 4 5
wrong passwords...
user36@netanya-students:~$
```

Ghidra

הגעתי לmain של chal1, ומצאתי את פקודת הcompare הבאה:

```
00101085 83 ff 03        CMP        EDI,0x3
```

```
RBX
EDI,0x3
LAB_00
RAX,qw
RDI,qw
```

	Hex	Decimal
byte	3h	3

כלומר האם יש 3 פרמטרים. (3 בבסיס 16 זה עדיין 3 בבסיס 10)

אבל הפרמטר הראשון תמיד יהיה שם הפונקציה אז צריך 2.

ומיד אחרי זה מופיעה פקודת JNZ

```
CMP        EDI,0x3
JNZ        LAB_001010c3
```

ובאסמבלי שזה נוגע לCMP וJNZ אם 2 הפרמטרים שווים, לא קורה כלום והקיד ממשיך "למטה", במקרה והפרמוטרים שונים הוא יקפוץ לLAB_001010c3.

ישר אחרי זה יש את פקודת הstrcmp הבאה

```
0010108a 48 8b 46 10     MOV        RAX,qword ptr [RSI + 0x10]
0010108e 48 8b 7e 08     MOV        RDI,qword ptr [RSI + 0x8]
00101092 48 89 c6        MOV        RSI,RAX
00101095 e8 d6 ff        CALL       <EXTERNAL>::strcmp
```

בstrcmp החלק המשמעותי הוא

```
                    **************************************************************
                    thunk int strcmp(char * __s1, char * __s2)
                      Thunked-Function: <EXTERNAL>::strcmp
     int              EAX:4          <RETURN>
     char *           RDI:8          __s1
     char *           RSI:8          __s2
                    <EXTERNAL>::strcmp                          XREF[1]:     main:00101095(c)
00101070 f3 0f 1e fa     ENDBR64
00101074 ff 25 56        JMP        qword ptr [->><EXTERNAL>::strcmp]          int strcmp(char * __s1, char * _...
         2f 00 00
```

כלומר משווים בין RDI וRSI והתוצאה מוחזרת לEAX

RSI מחזיק את הפרמטרים שהכנסנו, לכן מה שנכנסו ל RDI הוא הפרמטר השני (הראשון הוא שם הפונקציה, אז נכנס בRDI הפרמטר הראשון שכתבנו) ובRAX נכנס הפרמטר השלישי (כבייכול שני)

מעבירים RAX לRSI ומשווים

```
0010109a 89 c3           MOV        EBX,EAX
0010109c 85 c0           TEST       EAX,EAX
0010109e 74 15           JZ         LAB_001010b5
001010a0 48 8d 3d        LEA        RDI,[s_wrong_passwords..._0010207c]
         d5 0f 00 00
001010a7 e8 b4 ff        CALL       <EXTERNAL>::puts
         ff ff
```

אם הם שווים, לפי strcmp בEAX יהיה 0, לכן בשורה של הJZ תגרום לקפיצה ולא נקבל s_wrong_passwords.

אבל צריך 5 פרמטרים, אז איך נפתור ומה קרה פה?

אם נסתכל על _start

```
                    void processEntry _start(void)
void                  <VOID>          <RETURN>
undefined8            Stack[-0x10]:8 local_10                    XREF[1]:     001010f2(*)
                    _start                                   XREF[4]:     Entry Point(*), 00100018(*),
                                                                          001020bc, 001020e8(*)
                    start.S:57 (4)
001010e0 58             POP        RAX
001010e1 d0 e8          SHR        AL,0x1
001010e3 50             PUSH       RAX
```

בפונקציית _start מתבצעת הפקודה SHR AL, 0x1.

פקודה זו מבצעת הזזה של הביטים ימינה, שמשמעותה המתמטית היא **חלוקה ב-2** של כמות הארגומנטים (argc) לפני שהיא מגיעה לmain.

החישוב :אם נכניס 2 פרמטרים, התוכנית חישבה 1 = 2 / 3 (בחלוקת שלמים) (הוספנו 1 כי שם התכנית הוא נחשב לפרמטר). הפונקציה main בודקת אם הערך הוא 3, ולכן הבדיקה נכשלה!)

```
user36@netanya-students:~$ ./chal1 a a
please try again and make sure to give the correct amount of arguments („ˇ∪ˇ„)
user36@netanya-students:~$
```

הפתרון :כדי לעבור את הבדיקה, צריך לספק סה"כ **6 ארגומנטים** (שם התוכנית + 5 נוספים) (או אם מתיחסים להפעלת הפונקציה 5 פרמטרים), כך שהחישוב יהיה 3 = 2 / 6 וזה יספק את התנאי בmain.

```
user36@netanya-students:~$ ./chal1 a a doesntmatter doesntmatter alsodoesntmatter
correct! (ˇ^ ˇ ^ˇ)
user36@netanya-students:~$
```

<u>קלט פתרון:</u> 2 פרמטרים ראשונים שווים ו-3 שהם לא משנים.

Chal2

<u>Chal2</u>

אחרי analyze של Gihdra אנחנו נשלחים לentry, ואין main.

```
user36@netanya-students:~$ file /usr/local/share/data/assignment/chal2
/usr/local/share/data/assignment/chal2: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, Buil
dID[sha1]=2c9f906dded462cf90ebbe8bdce7269ea6d8e749, for GNU/Linux 3.2.0, stripped
user36@netanya-students:~$
```

אחרי הרצה של הפקודה file וקובץ chal2 אנחנו מגלים כי chal2 הוא stripped, כלומר השמות של הפונקציות בתוכו נדרסו לשמות אבסטרקטים (ניתן לראות בצילומי המסך)

אז בואו נסתכל על entry בdecompiler של Gihdra.

```
void processEntry entry(undefined8 param_1,undefined8 param_2)

{
  undefined1 auStack_8 [8];

  __libc_start_main(FUN_004023a0,param_2,&stack0x00000008,FUN_00403900,FUN_00403960,param_1,
                    auStack_8);
  do {
                    /* WARNING: Do nothing block with infinite loop */
  } while( true );
}
```

הפונקציה __libc_start_main מקבלת כפרמטר ראשון את פונקציית הmain.

אז נשנה את הlabel של הפרמטר הראשון לmain, ונמשיך לתוכו.

אנחנו נתקלים בהרבה קוד והרבה פונקציות, אז נעבור פונקציה פונקציה, נגלה מה היא עושה ונשנה לה את השם לשם המתאים (במקום השם הstripped)

```
local_c = 0;
                    /* try { // try from 004023b2 to 00402498 has
std::operator<<((ostream *)std::cout,"Password: ");
local_20 = 0;
local_28 = 0;
local_38 = 0;
FUN_00402930(&local_38);
```

נראה כי אנחנו מדפיסים על המסך את המחרוזת “Password: “

הנה הפונקציה הראשונה שאנו נתקלים בה, נכנס אליה:

```
void FUN_00402930(undefined8 *param_1)

{
  FUN_00402cb0(param_1);
  *param_1 = &PTR_FUN_00404088;
  return;
}
```

נכנס לפונקציה שבתוכה (הראשונה):

```
void FUN_00402cb0(undefined8 *param_1)

{
  *param_1 = &PTR___cxa_pure_virtual_004040a0;
  return;
}
```

ונכנס לכתובת שיש פה

```
                    PTR___cxa_pure_virtual_004040a0                XREF[2]:     FUN_00402cb0:00402cbe(*),
                                                                                FUN_00402cb0:00402cca(*)
004040a0 f0 21 40        addr        <EXTERNAL>::__cxa_pure_virtual
         00 00 00
         00 00
```

הפונקציה __cxa_pure_virtual היא פשוט מנגנון בטיחות. היא אומרת: "היי! מישהו ניסה להפעיל פונקציה שאין לה קוד (כי היא טהורה/אבסטרקטית). אני עוצרת את התוכנית עכשיו כדי למנוע קריסה לא מבוקרת."

כבר נחזור לשינוי השם של הפונקציה FUN_00402cb2.

```
void FUN_00402930(undefined8 *param_1)

{
  FUN_00402cb0(param_1);
  *param_1 = &PTR_FUN_00404088;
  return;
}
```

נכנס לקונציה השניה שיש פה שמופיעה ככתובת:

```
int FUN_00402970(undefined8 param_1)

{
  int local_14;
  undefined8 local_10;

  local_14 = 0;
  local_10 = param_1;
  std::istream::operator>>((istream *)std::cin,&local_14);
  return local_14;
}
```

פה אנחנו רואים קליטת קלט מהמשתמש והקלט נכנס לתוך local_14.

אז אנחנו נראה לפונקציה הזאת readInt.

```
void FUN_00402930(undefined8 *param_1)

{
  FUN_00402cb0(param_1);
  *param_1 = &PTR_readInt_00404088;
  return;
}
```

בחזרה לFUN_00402cb , נראה כי היא סתם שמה כמין בנאי אבסטראקטי.

והפונקציה המרכזית פשוט בונה את קלט הסיסמה מהמשתמש אז נשנה להם את השמות:

```
void passwordConstructor(undefined8 *param_1)

{
  abstructConstructor(param_1);
  *param_1 = &PTR_readInt_00404088;
  return;
}
```

Param_1 (הערך בו) הוא הכתובת לפונקצייה readInt.

נמשיך חזרה מmain:

```
passwordConstructor(&local_38);
local_2c = readInt(&local_38);
local_40 = 10;
piVar3 = (int *)FUN_004029b0(&local_2c,&local_40);
```

מסתבר שהפונקציה אחרי מה האחת שעברנו עליה עכשיו הייתה קליטת קלט,

אז יש לנו עכשיו בlocal_2c ובlocal_14 את קלט המשתמש.

ישנה פונקציה שלא עברנו עליה, נכנס אליה:

```
int * FUN_004029b0(int *param_1,int *param_2)

{
  int *local_10;

  local_10 = param_1;
  if (*param_2 < *param_1) {
    local_10 = param_2;
  }
  return local_10;
}
```

פונקציה ברורה: שמה בlocal_10 את הפרמטר הראשון ומשווה בין 2 הפרמטרים ואם הפרמטר השני קטן מהראשון נכניס בlocal_10 את הפרמטר השני ונחזיר אותו אחרת הפרמטר הראשון מוחזר דרכו, פונקציית החזרת מינימום. אז נקרא לה calcMin.

השימוש בקוד מראה כי פרמטר ראשון זה קלט המשתמש והשני זה המספר 10 כלומר, מחזירים את המספר הקטן בין הקלט למספר 10.

```
piVar3 = (int *)calcMin(&local_2c,&local_40);
local_3c = *piVar3;
local_48 = 5;
local_44 = 1;
uVar4 = FUN_004029f0(&local_28);
```

נראה כי המספר הקטן ביותר נכנס לתוך local_3c, ואנחנו רואים עוד פונקציה, נכנס אליה:

```
void FUN_004029f0(undefined8 param_1)

{
  FUN_00402cd0(param_1);
  return;
}
```

```
void FUN_00402cd0(undefined8 param_1)

{
  FUN_00402cf0(param_1);
  return;
}
```

```
void FUN_00402cf0(undefined8 param_1)

{
  FUN_00402d10(param_1);
  return;
}
```

```
undefined8 FUN_00402d10(undefined8 param_1)

{
  return param_1;
}
```

נראה כמו מסלול שפשוט בסופו של דבר מחזיר את הפרמטר שקיבלנו אשנה להם את השם ל(towardsReturnParam(number לפי סדר הופעתם, ולפונקציה שמחזירה את הפרמטר נקרא returnParam.

```
uVar4 = towardsReturnParam1(&local_28);
uVar5 = FUN_00402a10(&local_28);
```

נכנס לפונקציה הזאת:

```
void FUN_00402a10(undefined8 param_1)

{
  FUN_00402d20(param_1);
  return;
}

long FUN_00402d20(undefined8 param_1)

{
  long lVar1;

  lVar1 = towardsReturnParam3(param_1);
  return lVar1 + 10;
}
```

אנחנו רואים שיש פה עוד מסלול ובעזרת זה שנתנו שם לפונקציה שהייתה בדרך להחזרת הפרמטר, ניתן לראות שהמסלול פשוט מחזיר את הפרמטר פלוס 10.

נשנה את השם של הראשון לtowardsAdd10 והשני לadd10.

```
local_28 = 0;
local_38 = 0;
passwordConstructor(&local_38);
local_2c = readInt(&local_38);
local_40 = 10;
piVar3 = (int *)calcMin(&local_2c,&local_40);
local_3c = *piVar3;
local_48 = 5;
local_44 = 1;
uVar4 = towardsReturnParam1(&local_28);
uVar5 = towardsAdd10(&local_28);
local_50 = &local_48;
FUN_00402840(uVar4,uVar5,local_50);
```

ב2 הפונקציות ההחזרה הפרמטר היה כתובת לvariable שמחזיק את המספר 0.

אז ב uVar4 יש את הערך 0, בuVar5 יש את הערך 10, ובlocal_50 יש את הערך 5
(ניתין לראות שהוא הוגדר בצילום המסך)

נכנס לפונקציה שאנחנו רואים:

```
void FUN_00402840(undefined1 *param_1,undefined1 *param_2,undefined8 param_3)

{
  undefined1 uVar1;
  undefined1 *local_18;
  undefined8 local_10;

  local_10 = param_3;
  for (local_18 = param_1; local_18 != param_2; local_18 = local_18 + 1) {
    uVar1 = FUN_00402890(&local_10);
    *local_18 = uVar1;
  }
  return;
}
```

```
int FUN_00402890(undefined8 *param_1)

{
  char cVar1;

  cVar1 = FUN_004028c0(*param_1);
  return (int)(char)(cVar1 + 'a');
}
```

```
ulong FUN_004028c0(int *param_1)

{
  int iVar1;
  int local_18;
  int local_14;
  int *local_10;

  local_18 = *param_1;
  local_14 = param_1[1];
  iVar1 = *param_1;
  *param_1 = param_1[1];
  param_1[1] = iVar1 + param_1[1];
  local_10 = param_1;
  iVar1 = FUN_00402910(&local_18);
  return (long)iVar1 % 0x1a & 0xffffffff;
}
```

```
int FUN_00402910(int *param_1)

{
  return *param_1 + param_1[1];
}
```

נסתכל על הפונקציה הראשונה, בעצם מתייחסים אל param_1 כאל מערך בגודל 2, אז הפונקציה FUN_00402910 מחזירה סכום תא ראשון ושני במערך, נשנה לשם sumArrLen2.

אבל אם נסתכל בקוד נראה שהפרמטר בסוף הוא הפרמטר השלישי local_50 שמחזיק את הכתובת לlocal_48 שמחזיק את המספר 5, אבל נראה שגם local_44 אותחל עם ערך 1. אבל הוא נמצא בדיוק 4 בתים אחריו (השמות 48 ו-44 מייצגים את המרחק מתחתית המחסנית, ההפרש ביניהם הוא בדיוק 4, שזה הגודל של int)

לכן param_1 זה local_48 וparam_1[1] זה local_44.

נחזור לקוד של FUN_004028c0 ונראה כי הערך בparam_1 מקבל את הערך בכתובת param_1[1] אבל הערך המקורי בתוך param_1 נשמר ומשנים את הערך

בתוך param_1[1] לערך המקורי בparam_1 (לפני ששינו אותו) פלוס עצמו, ואז iVar1 מקבל את סכום הערכים המקוריים, (בעצם param_1[1] החדש), כי שמרו את הערכים המקוריים במשתנים עוקביים בstack: local_18 ו local_14 ושלחו לפונקציית הסכום שניתחנו.

נראה כמו צעד בסדרת פיבונאצי וגם עכשיו בiVar1 יש את המספר הבא בסדרת פיבונאצי (מהערכים המקוריים).

ובנוסף param_1 ו param_1[1] הם עכשיו הזוג הבא בסדרת פיבונאצי (שבparam_1 יש את הערך הישן של param_1[1])

וiVar1 עושים:

iVar1 % 0x1a & 0xffffffff

0x1a = 26 בבסיס 10 (המספר בבסיס 16)

ו - % זה פשוט מודולו

אז iVar2 מודולו 26

0xffffffff משמעותו 32 ביטים דולקים ו& זה סימן AND, אז לא קורה פה כלום עם החלק הזה כי הפעולה & (Bitwise AND) עם המספר הזה נועדה "לנקות" את המספר ולוודא שאנחנו מתייחסים רק ל-32 הביטים התחתונים שלו.

בשורה התחתונה עושים לסכום מודולו 26.

מה המטרה? המטרה היא להפוך את האיבר הבא בסדרת פיבונאצי לאות באנגלית, בשפה האנגלית 26 אותיות ומודולו 26 יתן מספר בין 0 ל25, נוכיח שזה בבירור אות כבר.

לפני זה נשנה את השם של הפונקציה לnextFibToLetter.

```
int FUN_00402890(undefined8 *param_1)

{
  char cVar1;

  cVar1 = nextFibToLetter(*param_1);
  return (int)(char)(cVar1 + 'a');
}
```

עכשיו חזרנו אחורה, וניתן לראות שמחזיקים את הערך בין 0 ל 25 ומוסיפים לו את התו 'a' ולבסוף ממירים את התוצאה לתו ואז מספר:

מה שיוצא מזה זה מס ASCII שמביא אות בין a ל z:

אם cVar שווה 0 אז יוחזר קוד הASCII של a.

אם cVar שווה 1 אז יוחזר קוד הASCII של b.

וכך האלה, אז נשנה את השם של הפונקציה לindexToASCII.

נחזור אחורה:

```
void FUN_00402840(undefined1 *param_1,undefined1 *param_2,undefined8 param_3)

{
  undefined1 uVar1;
  undefined1 *local_18;
  undefined8 local_10;

  local_10 = param_3;
  for (local_18 = param_1; local_18 != param_2; local_18 = local_18 + 1) {
    uVar1 = indexToASCII(&local_10);
    *local_18 = uVar1;
  }
  return;
}
```

פה אנחנו רואים לולאת for, ואם נתסכל מחדש על מה שניתן כ param_1 ו param_2 : ניתנו כתובות בהפרש 10.

בעצם נבנה פה מחרוזת של 10 תווים (ניתן לראות שזה מחרוזת על local_18 שכל פעם גדל ב1 והערך בו הוא תו.

אז נקרא לו passwordGenerator10.

כרגע ה10 תווים הראשונים של הסיסמה נמצאים בכתובת של local_28.

```
passwordGenerator10 (uVar4,uVar5,local_50);
memset(local_60,0,10);
for (local_64 = 0; local_64 < local_3c; local_64 = local_64 + 1) {
  local_70 = 0;
  FUN_00402a30(&local_70);
                    /* try { // try from 004024df to 0040261f has its CatchHandler @ 00402519 */
  uVar2 = FUN_00402a70(&local_70);
  puVar6 = (undefined1 *)FUN_00402ab0(local_60,(long)local_64);
  *puVar6 = uVar2;
}
```

פקודת הmemset לוקחת את המיקום בזיכרון מlocal_60 וממלא אותו באפסים לאורך 10 בתים.

והלולאה רצה עד מספר שהוא מקסימום 10 ומינימום קלט המשתמש. (אם נחזור על מה שהוכנס לlocal_3c בcalcMin)

מאפסים משתנה זמני local_70 ל 0 בכל סיבוב לולאה וקוראים לפונקציה הראשונה עם הכתובת שלו:

```
void FUN_00402a30(undefined8 *param_1)

{
  FUN_00402d40(param_1);
  *param_1 = &PTR_FUN_004040b8;
  return;
}
```

נכנס לפונקציה הראשונה:

```
void FUN_00402d40(undefined8 *param_1)

{
  *param_1 = &PTR___cxa_pure_virtual_004040d0;
  return;
}
```

נראה כמו בנאי אבסטראקטי כמו שראינו בהתחלה, אז נקרא לו AbstructConstructor2.

נחזור אחורה ונכנס לפונקציה השניה:

```
int FUN_00402a70(undefined8 param_1)

{
  char local_11;
  undefined8 local_10;

  local_11 = '\0';
  local_10 = param_1;
  std::operator>>((istream *)std::cin,&local_11);
  return (int)local_11;
}
```

לוקחים את local_11 מאפסים אותו לתו NULL, וקוראים לתוכו תו (כי local_11 הוגדר כchar), ומחזירים את הערך הASCII שלו.

אז נקרא לפונקציה readCharInner

ולפונקציה שקוראת לו readCharConstructor.

```
for (local_64 = 0; local_64 < local_3c; local_64 = local_64 + 1) {
  local_70 = 0;
  readCharConstructor(&local_70);
                    /* try { // try from 004024df to 0040261f has its CatchHandler @ 00402519 */
  uVar2 = readCharInner(&local_70);
  puVar6 = (undefined1 *)FUN_00402ab0(local_60,(long)local_64);
  *puVar6 = uVar2;
}
```

נמשיך ונראה שuVar2 מקבל את הקלט (תו)

נכנס לפוקנציה הבאה:

```
void FUN_00402ab0(undefined8 param_1,undefined8 param_2)

{
  FUN_00402d60(param_1,param_2);
  return;
}
```

```
long FUN_00402d60(long param_1,long param_2)

{
  return param_1 + param_2;
}
```

נראה שזאתי מחברת בין 2 long אז נקרא לה addLong

והשניה ממשת אותה, נראה כי local_64 הוא בעצם העקוב שלנו בזיכרון וכל פעם מתקדם ב-1 לכן ניתן להבין שזה הכתובת הבאה בזיכרון מהlocal_60 המקורי שבו 10 בתים של אפסים.

אז נקרא לפונקציה nextCell, ולבסוף נראה ששמים בכתובת הזאת את התו שנקרא, כלומר הלולאה הזאת קוראת את קלט הסיסמה עצמה מהמשתמש.

רגע אז בעצם מצאנו קלט ראשון של אורך סיסמה, קלט שני של נסיון סיסמה וגם את יוצר הסיסמה (עם פיבונאצי) אז אולי ההמשך של הקוד זה השוואות והחזרות שגיאה או הצלחה?א

אז ננסה לראות מה הסיסמה תיהיה עם קלט ראשון 10 (מספר):

אז ההתחלה הם 2 מספרים: 1 ו 5. ונוסחה של (מספר 1 + מספר 2) % 26

נוכחי: 1, 5	מס הבא:6	מודולו 26: 6	האות: g
נוכחי: 1, 6	מס הבא:7	מודולו 26: 7	האות: h
נוכחי: 6, 7	מס הבא:13	מודולו 26: 13	האות: n
נוכחי: 7, 13	מס הבא:20	מודולו 26: 20	האות: u
נוכחי: 13, 20	מס הבא:33	מודולו 26: 1	האות: h
נוכחי: 20, 33	מס הבא:53	מודולו 26: 7	האות: b
נוכחי: 33, 53	מס הבא:86	מודולו 26: 8	האות: i
נוכחי: 53, 86	מס הבא:139	מודולו 26: 9	האות: j
נוכחי: 86, 139	מס הבא:225	מודולו 26: 17	האות: r
נוכחי: 139, 225	מס הבא:364	מודולו 26: 0	האות: a

יצאנו עם הסיסמה:

ghnuhbijra

ננסה:

```
user36@netanya-students:~$ /usr/local/share/data/assignment/chal2
Password: 10
ghnuhbijra
Login successful.
```

הצלחנו!

נמשיך בכל זאת:

```
uVar4 = towardsReturnParam1 (local_60);
uVar5 = towardsAdd10 (local_60);
uVar7 = towardsReturnParam1 (&local_28);
uVar8 = towardsAdd10 (&local_28);
local_80 = FUN_00402ae0 (uVar4,uVar5,uVar7,uVar8);
```

uVar:

4 – קיבל את כתובת ההתחלה של סיסמה שניסינו להכניס

5- כתובת סוף של הסיסמה שניסינו

7- כתובת התחלה של סיסמה נכונה

8 – כתובת סיום של סיסמה נכונה

```
undefined1  [16]
FUN_00402ae0 (undefined8 param_1,undefined8 param_2,undefined8 param_3,undefined8 param_4)

{
  undefined1 auVar1 [16];

  FUN_00403090 ();
  auVar1 = FUN_00402fe0 (param_1,param_2,param_3,param_4);
  return auVar1;
}
```

```
void FUN_00403090 (void)

{
  return;
}
```

פונקציה שלא עושה כלום, נקרא לה nothing

```
undefined1  [16] FUN_00402fe0(long param_1,long param_2,long param_3,long param_4)

{
  byte local_41;
  long local_38;
  long local_30;
  long local_28;
  undefined1 local_20 [8];
  undefined1 local_18 [16];

  local_38 = param_3;
  local_30 = param_2;
  local_28 = param_1;
  while( true ) {
    local_41 = 0;
    if ((local_28 != local_30) && (local_41 = 0, local_38 != param_4)) {
      local_41 = FUN_004030a0(local_20,local_28,local_38);
    }
    if ((local_41 & 1) == 0) break;
    local_28 = local_28 + 1;
    local_38 = local_38 + 1;
  }
  FUN_004030d0(local_18,&local_28,&local_38);
  return local_18;
}
```

פה יש לנו לולאת while והפרמטרים הם גבולות של הסיסמה שניסינו להכניס והסיסמה הנכונה, אז כנראה הפונקציה מחזירה אם המחרוזות שוות.

הif הראשון בודק אם נשאר עוד מה לקרוא ולא הגענו לסוף הניחוש ולגם לא לסוף הסיסמה הנכונה.

ואז נכנסים לעוד פונקציה:

```
bool FUN_004030a0(undefined8 param_1,char *param_2,char *param_3)

{
  return *param_2 == *param_3;
}
```

הפונקציה משווה בין 2 תווים, ומחזירה 1 אם שווים ו 0 אם לא (Boolean) נקרא לה charIsEqual.

בחזרה ללולאה, יש שורת if שניה עם break שאומרת שאם התווים לא שווים לצאת מהלולאה, אחרת הקוד ממשיך ומקדמים את הindex-ים, כלומר התווים שווים.

מגיעים לעוד פונקציה:

```
void FUN_004030d0(undefined8 *param_1,undefined8 param_2,undefined8 param_3)

{
  undefined8 *puVar1;

  puVar1 = (undefined8 *)FUN_00403120(param_2);
  *param_1 = *puVar1;
  puVar1 = (undefined8 *)FUN_00403120(param_3);
  param_1[1] = *puVar1;
  return;
}
```

```
undefined8 FUN_00403120(undefined8 param_1)

{
  return param_1;
}
```

הפונקציה פשוט מחזירה מה את הפרמטר אז נקרא לה returnParam2.

לפי זה שכתוב undefined8 ומכניסים 2 פרמטרים מגודל כזה לתוכן param_1, וגם זה שהגדירו את local_18 כ: local_18 [16] אפשר להבין למה מוחזר מהפונקציה הגדולה FUN_00402fe0 מערך בגודל 16 בתים, אז הפונקציה FUN_004030d0 תקרא setTo16ByteArr.

והפונקציה הגדולה FUN_00402fe0 בעצם מחזירה את המיקום בו נמצא אי ההתאמה ב2 הסיסמאות: הזאת שהכנסנו והזאת שניסינו. נקרא לה findMismatch.

```
undefined1  [16]
FUN_00402ae0(undefined8 param_1,undefined8 param_2,undefined8 param_3,undefined8 param_4)

{
  undefined1 auVar1 [16];

  nothing();
  auVar1 = findMismatch(param_1,param_2,param_3,param_4);
  return auVar1;
}
```

בחזרה לכאן, נקרא לפונקציה הזאת returnMismatch.

נמשיך:

```
plVar9 = (long *)FUN_00402b50(local_80);
```

```
void FUN_00402b50(undefined8 param_1)

{
  FUN_00402d80(param_1);
  return;
}
```

```
undefined8 FUN_00402d80(undefined8 param_1)

{
  return param_1;
}
```

עוד פונקציות מבוך שמחזירות את הפרמטר שהביאו להן, נקרא להן returnParamAgain ונמספר (בשם).

נמשיך:

```
plVar9 = (long *)returnParamAgain(local_80);
lVar1 = *plVar9;
lVar10 = towardsAdd10(local_60);
if (lVar1 == lVar10) {
  plVar9 = (long *)FUN_00402b70(local_80);
  lVar1 = *plVar9;
  lVar10 = towardsAdd10(&local_28);
  if (lVar1 == lVar10) goto LAB_004026d8;
}
```

```
void FUN_00402b70(undefined8 param_1)

{
  FUN_00402d90(param_1);
  return;
}
```

```
long FUN_00402d90(long param_1)

{
  return param_1 + 8;
}
```

נקרא לה add8 ולאחת שמובילה אליה towardsAdd8.

```
plVar9 = (long *)returnParamAgain(local_80);
lVar1 = *plVar9;
lVar10 = towardsAdd10(local_60);
if (lVar1 == lVar10) {
  plVar9 = (long *)towardsAdd8(local_80);
  lVar1 = *plVar9;
  lVar10 = towardsAdd10(&local_28);
  if (lVar1 == lVar10) goto LAB_004026d8;
}
```

נראה כי lVar1 מקבל את המיקום שבו מצאו אי שוויון בנסיון (סיסמה) וlVar10 מקבל את הסוף של הנסיון (התחלה + 10), זה גם מצביע על כך שגודל הסיסמה חייב להיות 10 כי יעברו על 10 בתים ככה או ככה (כלומר אם מנסים להכניס סיסמה של פחות מ10 תווים ערכי זבל יקראו והסיסמה לא תעבוד).

ואז שואלים אם הגענו לסוף, כלומר הם שווים (הניחוש והסיסמה)

במקרה וכן מביאים את התא השני בreturnMismatch לlVar1, כי מתקדמים 8 בתים (towardsAdd8) כלומר מקבלים את הכתובת בה עצרנו בסיסמה הנכונה.

ולlVar10 מביאים את כתובת הסיום של הסיסמה. (local_28 מחזיק בכתובת ההתחלה של הסיסמה הנכונה)

והשורה האחרונה שולחת אותנו במקרה וגם הגענו לסוף שלה, כלומר הצלחה.

```
  std::allocator<char>::allocator();
                    /* try { // try from 00402659 to 00402670 has its CatchHandler @ 004026a4 */
  FUN_00402b90(local_a0,"Login unsuccessful.",local_a8);
                    /* try { // try from 00402676 to 00402681 has its CatchHandler @ 004026b2 */
  FUN_00402360(local_a0);
  std::__cxx11::string::~string(local_a0);
  std::allocator<char>::~allocator(local_a8);
LAB_004026d8:
  std::allocator<char>::allocator();
                    /* try { // try from 004026ee to 00402705 has its CatchHandler @ 00402739 */
  FUN_00402b90(local_c8,"Login successful.",local_d0);
                    /* try { // try from 0040270b to 00402716 has its CatchHandler @ 00402747 */
  FUN_00402360(local_c8);
  std::__cxx11::string::~string(local_c8);
  std::allocator<char>::~allocator(local_d0);
  return local_c;
}
```

זה נראה מפה פקודות החזרת הצלחה או כשלון, וניתן לראות שלאן שקופצים במקרה של הצלחה (LAB_004026d8) מחזיר "Login successful."

אז בשביל להיכנס:

Press Enter

ghnuhbijra

Press Enter

Chal3

כשמריצים את chal3 בהתחלה נותן לנו 3 אופציות:

ללחוץ 0: לשים שם משתמש

ללחוץ 1: לשים סיסמה

ללחוץ 2: להיכנס (ENTER אחרי)

אחרי הסתכלות בקוד ה main הבנתי כי אנחנו בעצם בstage0

```
                    undefined main()
     undefined        <UNASSIGNED>    <RETURN>
     undefined8         Stack[-0x10]:8 local_10                          XREF[2]:     001018bb(W),
                                                                                       001018da(R)
                    main                                          XREF[4]:     Entry Point(*),
                                                                               _start:001011d8(*), 00102410,
                                                                               001024fc(*)
001018a6 f3 0f 1e fa     ENDBR64
001018aa 55              PUSH       RBP
001018ab 48 89 e5        MOV        RBP,RSP
001018ae 48 83 ec 10     SUB        RSP,0x10
001018b2 64 48 8b        MOV        RAX,qword ptr FS:[0x28]
         04 25 28
         00 00 00
001018bb 48 89 45 f8     MOV        qword ptr [RBP + local_10],RAX
001018bf 31 c0           XOR        EAX,EAX
001018c1 48 8d 05        LEA        RAX,[s_welcome_agent_1337,_the_evil_gro_001023... = "welcome agent 1337, the evil ...
         60 0a 00 00
001018c8 48 89 c7        MOV        RDI=>s_welcome_agent_1337,_the_evil_gro_001023... = "welcome agent 1337, the evil ...
001018cb e8 50 f8        CALL       <EXTERNAL>::puts                                   int puts(char * __s)
         ff ff
001018d0 e8 96 fd        CALL       stage0                                             undefined stage0()
         ff ff
001018d5 b8 00 00        MOV        EAX,0x0
         00 00
001018da 48 8b 55 f8     MOV        RDX,qword ptr [RBP + local_10]
001018de 64 48 2b        SUB        RDX,qword ptr FS:[0x28]
         14 25 28
```

כלומר שלב 0, הראשון בעצם.

לקחתי את הlabels בstage0 והבאתי להם שמות משלי כדי להבין יותר טוב את הקוד ולעשות לעצמי סדר.

אז התסכלתי בקוד וראיתי תופעה מעניינת.

```
                    login                                          XREF[1]:     001016d3(j)
001017d7 48 8b 45 e0     MOV        RAX,qword ptr [RBP + username_input]
001017db 48 8d 15        LEA        RDX,[root_String]                              = 72h    r
         5b 0a 00 00
001017e2 48 89 d6        MOV        RSI=>root_String,RDX                           = 72h    r
001017e5 48 89 c7        MOV        RDI,RAX
001017e8 e8 73 f9        CALL       <EXTERNAL>::strcmp                             int strcmp(char * __s1, char * _...
         ff ff
001017ed 85 c0           TEST       EAX,EAX
001017ef 75 66           JNZ        Fail_Stage0+FreeUsernameMem_VULNERABILITY
001017f1 48 8d 05        LEA        RAX,[s_stage_1/2_of_logging_in_done..._00102288] = "stage 1/2 of logging in done....
         90 0a 00 00
001017f8 48 89 c7        MOV        RDI=>s_stage_1/2_of_logging_in_done..._0010228... = "stage 1/2 of logging in done....
001017fb e8 20 f9        CALL       <EXTERNAL>::puts                               int puts(char * __s)
         ff ff
00101800 48 8b 45 e8     MOV        RAX,qword ptr [RBP + password_input]
00101804 48 8d 15        LEA        RDX,[root_String]                              = 72h    r
         32 0a 00 00
0010180b 48 89 d6        MOV        RSI=>root_String,RDX                           = 72h    r
0010180e 48 89 c7        MOV        RDI,RAX
00101811 e8 4a f9        CALL       <EXTERNAL>::strcmp                             int strcmp(char * __s1, char * _...
         ff ff
00101816 85 c0           TEST       EAX,EAX
00101818 75 0f           JNZ        success_username
0010181a 48 8d 05        LEA        RAX,[s_the_username_and_password_cannot_001022... = "the username and password can...
         87 0a 00 00
00101821 48 89 c7        MOV        RDI=>s_the_username_and_password_cannot_001022... = "the username and password can...
00101824 e8 f7 f8        CALL       <EXTERNAL>::puts                               int puts(char * __s)
         ff ff
```

```
                    root_String                                    XREF[6]:     stage0:00101744(*),
                                                                                stage0:0010174b(*),
                                                                                stage0:001017db(*),
                                                                                stage0:001017e2(*),
                                                                                stage0:00101804(*),
                                                                                stage0:0010180b(*)
0010223d 72          ??         72h    r
0010223e 6f          ??         6Fh    o
0010223f 6f          ??         6Fh    o
00102240 74          ??         74h    t
00102241 00          ??         00h
00102242 00          ??         00h
00102243 00          ??         00h
00102244 00          ??         00h
00102245 00          ??         00h
00102246 00          ??         00h
00102247 00          ??         00h
```

בעצם משווים את הקלט למחרוזת "root", וזוהי המחרוזת הנכונה לusername.

אבל אם נסתכל על קליטת username:

```
00101717 bf 64 00        MOV        EDI,0x64
         00 00
0010171c e8 5f fa        CALL       <EXTERNAL>::malloc                          void * malloc(size_t __size)
         ff ff
00101721 48 89 45 e0     MOV        qword ptr [RBP + username_input],RAX
00101725 48 8b 45 e0     MOV        RAX,qword ptr [RBP + username_input]
00101729 48 89 c6        MOV        RSI,RAX
0010172c 48 8d 05        LEA        RAX,[scanning_string]                       = 25h    %
         bd 09 00 00
00101733 48 89 c7        MOV        RDI=>scanning_string,RAX                    = 25h    %
00101736 b8 00 00        MOV        EAX,0x0
         00 00
0010173b e8 60 fa        CALL       <EXTERNAL>::__isoc99_scanf                  undefined __isoc99_scanf()
         ff ff
00101740 48 8b 45 e0     MOV        RAX,qword ptr [RBP + username_input]
00101744 48 8d 15        LEA        RDX,[root_String]                           = 72h    r
         f2 0a 00 00
0010174b 48 89 d6        MOV        RSI=>root_String,RDX                        = 72h    r
0010174e 48 89 c7        MOV        RDI,RAX
00101751 e8 0a fa        CALL       <EXTERNAL>::strcmp                          int strcmp(char * __s1, char * _...
         ff ff
00101756 85 c0           TEST       EAX,EAX
00101758 0f 85 30        JNZ        username_loop_input
         01 00 00
0010175e 48 8d 05        LEA        RAX,[s_you_are_not_allowed_to_login_as_a_00102... = "you are not allowed to login ...
         e3 0a 00 00
00101765 48 89 c7        MOV        RDI=>s_you_are_not_allowed_to_login_as_a_00102... = "you are not allowed to login ...
00101768 e8 b3 f9        CALL       <EXTERNAL>::puts                            int puts(char * __s)
         ff ff
0010176d 48 8b 45 e0     MOV        RAX,qword ptr [RBP + username_input]
00101771 48 89 c7        MOV        RDI,RAX
00101774 e8 87 f9        CALL       <EXTERNAL>::free                            void free(void * __ptr)
         ff ff
00101779 48 c7 45        MOV        qword ptr [RBP + username_input],0x0
         e0 00 00
         00 00
00101781 e9 08 01        JMP        username_loop_input
         00 00
```

הערה: זה לא כל החלק של קליטת המשתמש, לא הכללתי את ההדפסה שאומר להכניס שם משתמש.

ניתן לראות כאן כי מקצים זיכרון לusername, והוא נשמר במקום ששמתי לו label כ: username_input, אבל משווים אותו גם למחרוזת root, ובמקרה של שוויון מודפס איסור (לא שגיאה אבל מין חסימה), ועושים free למקום בזיכרון ומאפסים את המצביע לשם המשתמש, ואז חוזרים משם לקליטה הראשית (שאשפר ללחוץ 0 או 1 או 2).

כלומר אנחנו בבעיה, השם משתמש שאנחנו צריכים להכניס בשביל לעבור את stage0 לא ניתן להכנסה קקלט, נחזור לבעיה, קודם כל נסתכל על מה תריכה להיות הסיסמה:

צילום המסך המופיע מתחת לשורה הזאת הוא הקצאת המקום בזיכרון והמצביע להיכן שקלט הסיסמה נמצא. (הוא נמצא ב[RBP + password_input] שהוא מקום פיזי בזיכרון, password_input כהoffset. כנל לגבי username_input.)

```
                    enter_password                                  XREF[1]:     001016e9(j)
00101786 48 8d 05        LEA        RAX,[s_enter_ur_password:_00102272]          = "enter ur password: "
         e5 0a 00 00
0010178d 48 89 c7        MOV        RDI=>s_enter_ur_password:_00102272,RAX       = "enter ur password: "
00101790 b8 00 00        MOV        EAX,0x0
         00 00
00101795 e8 a6 f9        CALL       <EXTERNAL>::printf                           int printf(char * __format, ...)
         ff ff
0010179a 48 8b 05        MOV        RAX,qword ptr [stdout]
         6f 28 00 00
001017a1 48 89 c7        MOV        RDI,RAX
001017a4 e8 e7 f9        CALL       <EXTERNAL>::fflush                           int fflush(FILE * __stream)
         ff ff
001017a9 bf 64 00        MOV        EDI,0x64
         00 00
001017ae e8 cd f9        CALL       <EXTERNAL>::malloc                           void * malloc(size_t __size)
         ff ff
001017b3 48 89 45 e8     MOV        qword ptr [RBP + password_input],RAX
001017b7 48 8b 45 e8     MOV        RAX,qword ptr [RBP + password_input]
001017bb 48 89 c6        MOV        RSI,RAX
001017be 48 8d 05        LEA        RAX,[scanning_string]                        = 25h    %
         2b 09 00 00
001017c5 48 89 c7        MOV        RDI=>scanning_string,RAX                     = 25h    %
001017c8 b8 00 00        MOV        EAX,0x0
         00 00
001017cd e8 ce f9        CALL       <EXTERNAL>::__isoc99_scanf                   undefined __isoc99_scanf()
         ff ff
001017d2 e9 bb 00        JMP        loop_again_on_input
         00 00
```

וכאשר לוחצים 2 כדי להיכנס: (מסומנת שורת ההשוואה)

```
0010169e 48 8d 05        LEA        RAX,[s_0:_enter_username,_1:_enter_pass_001021... = "0: enter username, 1: enter
         53 0b 00 00
001016a5 48 89 c7        MOV        RDI=>s_0:_enter_username,_1:_enter_pass_001021... = "0: enter username, 1: enter
001016a8 b8 00 00        MOV        EAX,0x0
         00 00
001016ad e8 8e fa        CALL       <EXTERNAL>::printf                                int printf(char * __format, ..
         ff ff
001016b2 48 8b 05        MOV        RAX,qword ptr [stdout]
         57 29 00 00
001016b9 48 89 c7        MOV        RDI,RAX
001016bc e8 cf fa        CALL       <EXTERNAL>::fflush                                int fflush(FILE * __stream)
         ff ff
001016c1 e8 aa fa        CALL       <EXTERNAL>::getchar                               int getchar(void)
         ff ff
001016c6 88 45 df        MOV        byte ptr [RBP + local_29],AL
001016c9 0f be 45 df     MOVSX      EAX,byte ptr [RBP + local_29]
001016cd 83 e8 30        SUB        EAX,0x30
001016d0 83 f8 02        CMP        EAX,0x2
001016d3 0f 84 fe        JZ         login
         00 00 00
001016d9 83 f8 02        CMP        EAX,0x2
001016dc 0f 8f b0        JG         loop_again_on_input
         01 00 00
001016e2 85 c0           TEST       EAX,EAX
001016e4 74 0e           JZ         enter_username
001016e6 83 f8 01        CMP        EAX,0x1
001016e9 0f 84 97        JZ         enter_password
         00 00 00
001016ef e9 9e 01        JMP        loop_again_on_input
         00 00
```

בודקים אם הusername נכון: (מסומנת שורת הstrcmp)

```
                    login                                          XREF[1]:     001016d3(j)
001017d7 48 8b 45 e0     MOV        RAX,qword ptr [RBP + username_input]
001017db 48 8d 15        LEA        RDX,[root_String]                                = 72h    r
         5b 0a 00 00
001017e2 48 89 d6        MOV        RSI=>root_String,RDX                             = 72h    r
001017e5 48 89 c7        MOV        RDI,RAX
001017e8 e8 73 f9        CALL       <EXTERNAL>::strcmp                               int strcmp(char * __s1, char * _...
         ff ff
001017ed 85 c0           TEST       EAX,EAX
001017ef 75 66           JNZ        Fail_Stage0+FreeUsernameMem_VULNERABILITY
001017f1 48 8d 05        LEA        RAX,[s_stage_1/2_of_logging_in_done..._00102288] = "stage 1/2 of logging in done....
         90 0a 00 00
001017f8 48 89 c7        MOV        RDI=>s_stage_1/2_of_logging_in_done..._0010228... = "stage 1/2 of logging in done....
001017fb e8 20 f9        CALL       <EXTERNAL>::puts                                 int puts(char * __s)
         ff ff
00101800 48 8b 45 e8     MOV        RAX,qword ptr [RBP + password_input]
00101804 48 8d 15        LEA        RDX,[root_String]                                = 72h    r
         32 0a 00 00
0010180b 48 89 d6        MOV        RSI=>root_String,RDX                             = 72h    r
0010180e 48 89 c7        MOV        RDI,RAX
00101811 e8 4a f9        CALL       <EXTERNAL>::strcmp                               int strcmp(char * __s1, char * _...
         ff ff
00101816 85 c0           TEST       EAX,EAX
00101818 75 0f           JNZ        success_username
0010181a 48 8d 05        LEA        RAX,[s_the_username_and_password_cannot_001022... = "the username and password can...
         87 0a 00 00
00101821 48 89 c7        MOV        RDI=>s_the_username_and_password_cannot_001022... = "the username and password can...
00101824 e8 f7 f8        CALL       <EXTERNAL>::puts                                 int puts(char * __s)
         ff ff
```

ואז משווים לסיסמה הרצויה: מחרוזת IAMROOT1337.

```
                    success_username                                XREF[1]:     00101818(j)
00101829 48 8b 45 e8     MOV        RAX,qword ptr [RBP + password_input]
0010182d 48 8d 15        LEA        RDX,[correct_password_string]                  = "IAMROOT1337"
         a2 0a 00 00
00101834 48 89 d6        MOV        RSI=>correct_password_string,RDX               = "IAMROOT1337"
00101837 48 89 c7        MOV        RDI,RAX
0010183a e8 21 f9        CALL       <EXTERNAL>::strcmp                             int strcmp(char * __s1, char * _...
         ff ff
0010183f 85 c0           TEST       EAX,EAX
00101841 75 14           JNZ        Fail_Stage0+FreeUsernameMem_VULNERABILITY
00101843 48 8d 05        LEA        RAX,[s_stage_2/2_of_logging_in_done..._001022e8] = "stage 2/2 of logging in done....
         9e 0a 00 00
0010184a 48 89 c7        MOV        RDI=>s_stage_2/2_of_logging_in_done..._001022e... = "stage 2/2 of logging in done....
0010184d e8 ce f8        CALL       <EXTERNAL>::puts                               int puts(char * __s)
         ff ff
00101852 e8 46 fd        CALL       stage1                                         undefined stage1()
         ff ff
```

```
                    correct_password_string                         XREF[2]:     stage0:0010182d(*),
                                                                                 stage0:00101834(*)
001022d6 49 41 4d        ds         "IAMROOT1337"
         52 4f 4f
         54 31 33 ...
001022e2 00              ??         00h
001022e3 00              ??         00h
001022e4 00              ??         00h
001022e5 00              ??         00h
001022e6 00              ??         00h
001022e7 00              ??         00h
```

וניתן לראות שבמקרה ששני שלבי שם המשתמש והסיסמה עוברים בהצלחה כלומר הכניסו את הקלטים הנכונים, עוברים לstage1.

אבל ניתן לראות שגם בקוד ששם המשתמש נכון וגם בנסיון הlogin, אנחנו קופצים ל:Fail_Stage0+FreeUsernameMem_VULNERABILITY, label שנתתי לכישלון התחברות ששם מופיעה גם החולשה.

```
                    Fail_Stage0+FreeUsernameMem_VULNERABILITY        XREF[2]:     001017ef(j), 00101841(j)
00101857 48 8d 05        LEA        RAX,[s_please_try_again_as_an_admin!_00102308]   = "please try again as an admin!"
         aa 0a 00 00
0010185e 48 89 c7        MOV        RDI=>s_please_try_again_as_an_admin!_00102308,... = "please try again as an admin!"
00101861 e8 ba f8        CALL       <EXTERNAL>::puts                                  int puts(char * __s)
         ff ff
00101866 48 83 7d        CMP        qword ptr [RBP + username_input],0x0
         e0 00
0010186b 74 0c           JZ         free_password_mem_VULNERABILITY
0010186d 48 8b 45 e0     MOV        RAX,qword ptr [RBP + username_input]
00101871 48 89 c7        MOV        RDI,RAX
00101874 e8 87 f8        CALL       <EXTERNAL>::free                                  void free(void * __ptr)
         ff ff

                    free_password_mem_VULNERABILITY                  XREF[1]:     0010186b(j)
00101879 48 83 7d        CMP        qword ptr [RBP + password_input],0x0
         e8 00
0010187e 74 11           JZ         password_loop_input
00101880 48 8b 45 e8     MOV        RAX,qword ptr [RBP + password_input]
00101884 48 89 c7        MOV        RDI,RAX
00101887 e8 74 f8        CALL       <EXTERNAL>::free                                  void free(void * __ptr)
         ff ff
0010188c eb 03           JMP        password_loop_input
```

כאן מופיעה החולשה, לכאן בעצם "נזרק" כל פעם שננסה להיכנס למערכת (בלי לכתוב root כשם משתמש) ולא משנה הסיסמה, גם אם היא IAMROOT1337.

<u>החולשה:</u>

עושים free למיקום של הoffset של שם המשתמש ושל הסיסמה (username_input, password_input) אבל **לא מאפסים את התוכן שלהם**, כלומר הם עדיין מצביעים למקומות שהוקצו להם בזיכרון למרות שהמקומות האלה עכשיו מוגדרים כמשוחררים.

הערה: המיקום הפיזי של שם המשתמש והסיסמה הוא:

[RBP + username_input] ו- [RBP + password_input].

<u>ניצול:</u>

מערכת ההפעלה Linux מנהלת את הזיכרון המשוחרר בשיטת LIFO:

(Last-In, First-Out - האחרון שנכנס הוא הראשון שיוצא).

נניח לשם המשתמש הוקצה קטע A ולסיסמה הוקצה קטע B. ושניהם הוקצו עם אותו זיכרון בגודל 100 בתים.

```
         -- --
00101717 bf 64 00        MOV        EDI,0x64
         00 00
0010171c e8 5f fa        CALL       <EXTERNAL>::malloc                                void * malloc(size_t __size)
         ff ff
00101721 48 89 45 e0     MOV        qword ptr [RBP + username_input],RAX
00101725 48 8b 45 e0     MOV        RAX,qword ptr [RBP + username_input]
```

```
001017a9 bf 64 00        MOV        EDI,0x64
         00 00
001017ae e8 cd f9        CALL       <EXTERNAL>::malloc                    void * malloc(size_t __size)
         ff ff
001017b3 48 89 45 e8     MOV        qword ptr [RBP + password_input],RAX
001017b7 48 8b 45 e8     MOV        RAX,qword ptr [RBP + password_input]
```

ואז נניח והגענו לכשל בlogin (בלי לקרוא לשם המשתמש root, כי במקרה זה הקטע של שם המתשמש והזיכרון בו משוחררים באופן אופטימלי וגם זה מאפס אותו כך שאם ננסה להיכנס נקבל שגיאת segmentation) אז עושים קודם כל free לזיכרון שהוקצה לשם המשתמש (קטע A) ואז לזיכרון שהוקצה לסיסמה (קטע B)

אז עכשיו גם מפני שהוקצה להם אותו גודל בזיכרון: הקצאה הבאה לאותו גודל 100 בתים, סל הLIFO נראה כך:

A <- B (B יצא קודם ואז A, B בראש הרשימה)

לשם הפשטות נקרא למצביע על שם המשתמש user_ptr ולמצביע לסיסמה pwd_ptr.

אז עכשיו נלחץ 1 ונתן קקלט כבייכול סיסמה, הסיסמה לא משנה (לדוגמה adsads), אז עכשיו יוקצה קטע B לסיסמה והכתובת תשמר בpwd_ptr.

עכשיו הקטע הבא שיצא הוא קטע A, שuser_ptr עדיין מצביע אליו.

אז נלחץ שוב פעם 1 ונכתוב root, אז עכשיו קטע A מוקצה לpwd_ptr ומחזיק בתוכו את המחרוזת root, אבל גם user_ptr מצביע לשם והנה עכשיו עקפנו את החסימה לקלט root הנכון.

נלחץ שוב 1 ונתן קקלט את הסיסמה הנכונה: IAMROOT1337, והוקצה קטע חדש לגמרי (אפשר לקרוא לו C) שpwd_ptr מצביע אליו שמכיל א הסיסמה הנכונה.

נלחץ 2 ונעשה אנטר כדי להיכנס והנה עברנו לstage1.

```
user36@netanya-students:~$ /usr/local/share/data/assignment/chal3
welcome agent 1337, the evil group 0xL0CCEDC0DE has stolen 1337 BILLION dollars, you MUST get it back...
for your first take you must hack into the admin account
0: enter username, 1: enter password, 2: login> 1
enter ur password: jjjj
0: enter username, 1: enter password, 2: login> 0
enter ur username: aaaa
0: enter username, 1: enter password, 2: login> 2
please try again as an admin!
0: enter username, 1: enter password, 2: login> 1
enter ur password: adsads
0: enter username, 1: enter password, 2: login> 1
enter ur password: root
0: enter username, 1: enter password, 2: login> 1
enter ur password: IAMROOT1337
0: enter username, 1: enter password, 2: login> 2
stage 1/2 of logging in done...
stage 2/2 of logging in done...
good job agent 1337! you must now use ur secret device to change the code to unlock the door! the current code is ABCD and you must change it!
```